

Sistemas Operativos

Práctico 5: Gestión de la memoria y memoria virtual

1. Determinar los espacios de memoria (incluyendo swap) que están usando los procesos de su sistema. Ayuda: Ver los comandos `top`, `free`, `smem` y `swapon`.
2. Mostrar la evolución de un heap de tamaño de 1MB gestionado con un sistema buddy con bloques de tamaño entre 2^5 y 2^{10} con la secuencia de las siguientes operaciones:
 - a. `ptr1 = malloc(20)`
 - b. `ptr2 = malloc(50)`
 - c. `ptr3 = malloc(1010)`
 - d. `free(ptr1)`
 - e. `malloc(64)`
3. Hacer un programa de usuario en Linux o MS-Windows que permita analizar en qué direcciones lógicas se encuentran las áreas de código, datos, código, heap y stack.
4. Una arquitectura con un espacio de direcciones de 16 bits. La memoria se particiona lógicamente con páginas de 1KB.
 - a. Determinar la capacidad máxima de la memoria en KB).
 - b. Determinar la cantidad de páginas de la memoria.
 - c. Proponer una representación de la tabla de páginas y el formato de cada entrada (page table entry).
 - d. El espacio de direcciones de un proceso es desde 0 a su tamaño máximo (incluyendo la pila). Mostrar la tabla de páginas de un proceso que usa 1800 bytes de código, a continuación, 600 bytes de datos y 1KB para su pila. Entre los datos y la pila debe haber una página de guarda para detectar un posible *stack overflow*.
5. En un SO en una arquitectura de 32 bits con paginado. Determine la cantidad de entradas de tablas de páginas (*pte*) y cantidad de memoria total requeridas para completar un espacio de direcciones de 4GB.
 - a. Con páginas de 4KB de tamaño.
 - b. Con páginas de 4MB de tamaño.
6. Representar la tabla de páginas de un proceso
7. Dos procesos comparten un área de memoria de 2 páginas. En el primer proceso esa área de memoria se mapea a partir de la dirección 0xA000. En el segundo desde 0xF000. Mostrar con un diagrama el layout de memoria de cada proceso y las tablas de páginas. Asuma que las páginas son de 4KB.
8. Compilar y ejecutar (en Linux) el archivo `mmapfile.c` dado.
 - a. El archivo creado (`mmapfile.txt`) ocupa poco más de 4096 bytes. Porqué el acceso a `data + 7000` no causa un segmentation fault?
 - b. Modificar el programa para hacer el mapeo del archivo para lectura y escritura y que las modificaciones queden almacenadas en el archivo automáticamente al liberar el mapping. ¿Cómo hace esto el SO?
 - c. Verificar el tamaño del archivo luego que finalice el programa.

9. Compilar y ejecutar los programas dados `shmwriter.c` y `shmreader.c` (en ese orden) para poder observar cómo funciona la API shared memory (IPC de UNIX).
 - a. Correr nuevamente el programa `shredder` y ver qué ocurre.
 - b. Comentar la línea que destruye la shm en `shmreader.c`. Correr el programa `shmwriter` y luego correr varias veces `shmreader`.
 - c. Modificar ambos programas para que escriban y lean datos en la dirección lógica `str+500`.